ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA



Dia, Fecha:	Martes, 01/09/2026
Hora de inicio:	12:20 pm

Laboratorio de Programación de Computadoras 1 Sección N

Luis Vejo

Iniciamos 12:25 pm

SEMANA 8: SQL INTERMEDIO

Consultas Avanzadas y Análisis de Datos

Laboratorio de Programación de Computadoras 1 - Sección N

1. Introducción: ¿Qué haremos hoy?

- **Evolucionar:** Pasaremos del manejo básico de datos (DDL y DML) al análisis y resumen de información (DQL Avanzado).
- **Agrupar y Filtrar:** Aprenderemos a consolidar datos a nivel macro utilizando funciones de agregación.
- **Vistas (Views):** Conoceremos cómo estructurar y guardar consultas recurrentes de manera eficiente en la base de datos.

2. Valor de la Semana

> La responsabilidad es el compromiso de cumplir con las tareas asignadas, asumiendo las consecuencias y tomando decisiones conscientes.
>
> — La Responsabilidad

Aplicación: Se refleja al garantizar la integridad de los datos en consultas complejas, y en la adecuada organización del tiempo para planificar y resolver los laboratorios semanales.

3. Conocimientos: Más allá del SELECT básico

¿Por qué necesitamos SQL Intermedio?

Las bases de datos en la vida real empresarial tienen miles o millones de registros. Una consulta básica como SELECT * deja de ser útil para la toma de decisiones.

- **El Objetivo:** Necesitamos transformar "Datos" crudos en "Información" procesada (reportes, estadísticas, resúmenes de venta).
- **La Solución:** El motor relacional está optimizado para realizar cálculos matemáticos masivos antes de entregar la data a la aplicación.



Funciones de Agregación



COUNT()

Cuenta el número total de filas que coinciden con una condición especificada.



SUM()

Suma los valores numéricos de una columna en particular.



AVG()

Calcula el promedio (la media) matemático de una columna numérica.



MAX() / MIN()

Encuentran el valor más alto o el más bajo dentro de un conjunto de datos.

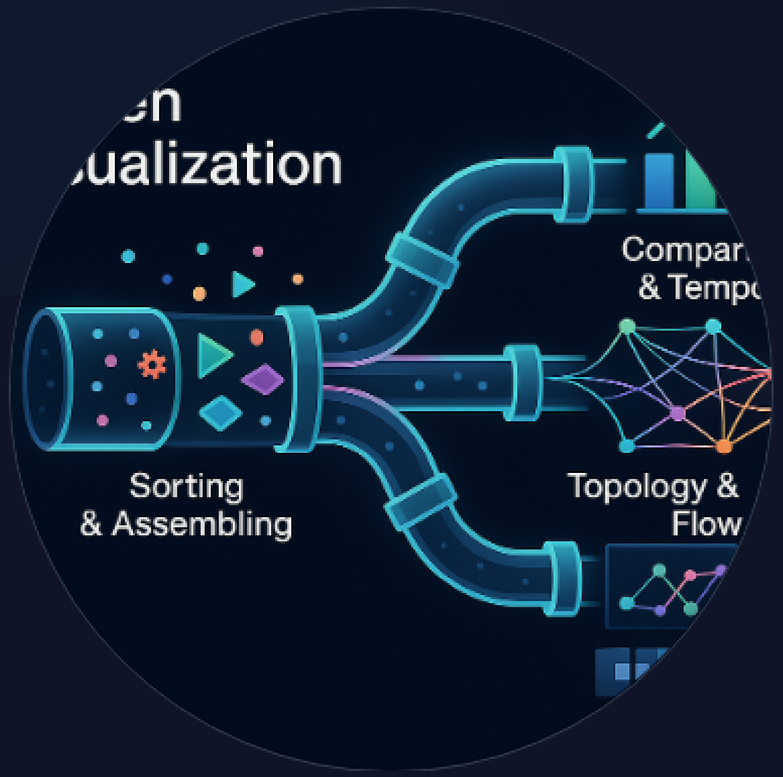
Ejemplo de Agregación en Acción

Aplicando las funciones sobre nuestra base de datos de la "Tienda en Línea":

```
SELECT COUNT(id_producto) AS Total_Productos, SUM(stock) AS Articulos_En_Bodega, AVG(precio) AS Precio_Promedio, MAX(precio) AS Producto_Mas_Caro FROM Productos;
```

ℹ️ Nota: Esta consulta devolverá una única fila resumiendo toda la tabla de productos.

Agrupando Datos (GROUP BY)



La cláusula GROUP BY

El **GROUP BY** permite agrupar filas que tienen los mismos valores en filas de resumen.

Se utiliza casi de manera obligatoria en conjunto con las funciones de agregación. Sin esta cláusula, las funciones aplicarían a toda la tabla de forma global.

Analogía: "Dime cuántos productos hay en total, pero agrúpalos POR cada categoría".

Agrupación en la Práctica

Combinando el JOIN de la semana pasada con la nueva cláusula GROUP BY:

```
SELECT c.nombre AS Categoria, COUNT(p.id_producto) AS Cantidad_Productos, SUM(p.stock) AS Total_Inventario FROM Productos p JOIN Categorias c ON p.id_categoria = c.id_categoria
GROUP BY
    c.nombre;
```

Filtrando Grupos: WHERE vs HAVING

WHERE

La cláusula WHERE se ejecuta **antes** de la agrupación.

Se utiliza para filtrar registros individuales de las tablas originales antes de que el motor de SQL realice cualquier cálculo o resumen.

HAVING

La cláusula HAVING se ejecuta **después** de la agrupación.

Fue añadida a SQL porque el WHERE no puede utilizarse con funciones de agregación. Sirve para filtrar los resultados de los grupos ya consolidados.

Ejemplo de HAVING

Mostrar un reporte, pero únicamente de aquellas categorías que tengan **más de 1 producto** asignado:

```
SELECT c.nombre AS Categoria, COUNT(p.id_producto) AS Cantidad_Productos FROM Productos p
JOIN Categorias c ON p.id_categoria = c.id_categoria GROUP BY c.nombre HAVING
COUNT(p.id_producto) > 1;
```

Vistas (Views)

Guardando nuestras consultas

Una **Vista** es una tabla virtual basada en el conjunto de resultados de una consulta SQL estructurada.

Contiene filas y columnas, igual que una tabla real. Los campos de una vista provienen de campos de una o más tablas reales de la base de datos.

Ventaja principal: Nos evita tener que reescribir un JOIN complejo o un GROUP BY cada vez que necesitemos el reporte.



¿Cómo se crea un VIEW?

Guardando nuestro reporte de inventario directamente en el servidor:

```
CREATE VIEW Vista_Inventario_Categorias AS SELECT c.nombre AS Categoria,
COUNT(p.id_producto) AS Cantidad FROM Productos p JOIN Categorias c ON p.id_categoria =
c.id_categoria GROUP BY c.nombre;
-- Para consultarlo en el futuro, simplemente ejecutamos:
SELECT * FROM Vista_Inventario_Categorias;
```

4. Reflexión y cierre

Hemos pasado de simplemente almacenar datos a tener la capacidad de analizarlos y extraer reportes.

Asignación: Resolver el cuestionario práctico de SQL intermedio en UEDI.

Cualquier duda técnica se canalizará exclusivamente por el Foro Oficial. | Auxiliar: Luis Vejo

ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

ECYS | DTT
DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Nombre de la actividad:	Hoja de Trabajo # 2
Cantidad de participantes:	14
Doy fe que esta actividad está planificada en dtt (Sí/No):	Si

Hora de inicio:	13:30
Hora de fin:	13:50
Duración (min):	20min

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

202401286 - Rene Casasola
Cristhian Giovanni Garcia Tumax 202300529
202100307-Josseline Marisol Ramírez Calderón
202308426 - Handery Isabel Cardona Almaraz
201602495 - José Emanuel Salazar Cano
201612281 - Julio Alfredo Cruz Gonzalez
Angie Paola Herrera Vargas Machuca - 202102606
Daniel Fernando Juarez Andrino 202004436
Santiago Benjamin Velásquez Chávez - 202502940

elias abraham gonzalez solval 202307739
Tiffany Michelle Palacios Perez 202208685
202203909-Moises Alejandro Orozco Cárcamo
202307514 - Lilian Esther Jimenez Games
202110311- Wendy Karina Mijangos Morales